



ICT2216/ICT2516C

Secure Software Development

Secure Software Design x02

Raymond Chan & Tram Truong-Huu

Agenda

- Design with Core Security Concepts
 - CIA
 - Authentication, Authorization and Accountability
- Design with Security Design Principles
 - Security architecture
 - Secure Software Design
 - Secure Agile Development

SECURE SOFTWARE DESIGN

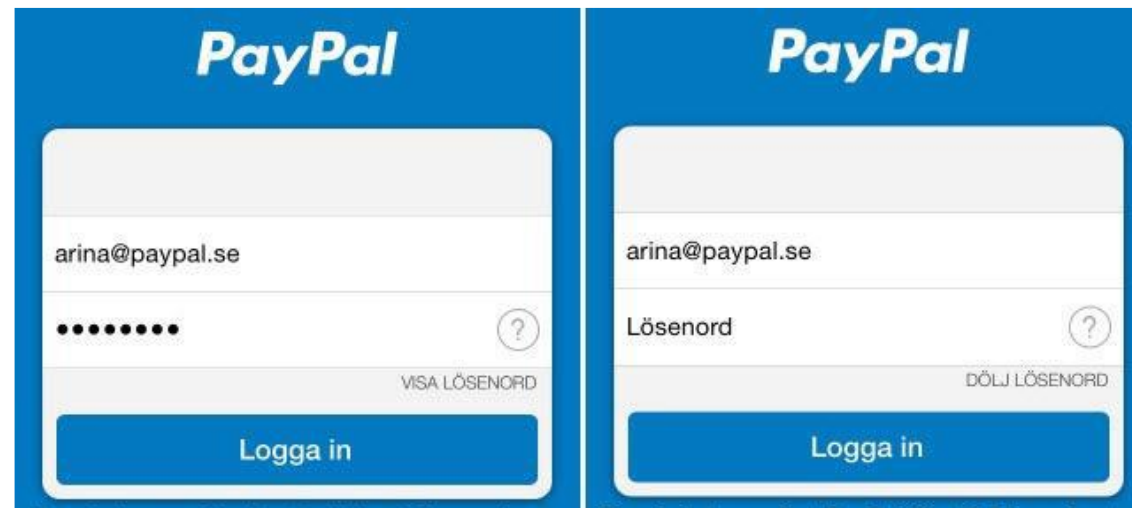
Secure Software Design

- Confidentiality Design
- Integrity Design
- Availability Design
- Authentication Design
- Authorization Design
- Accountability Design



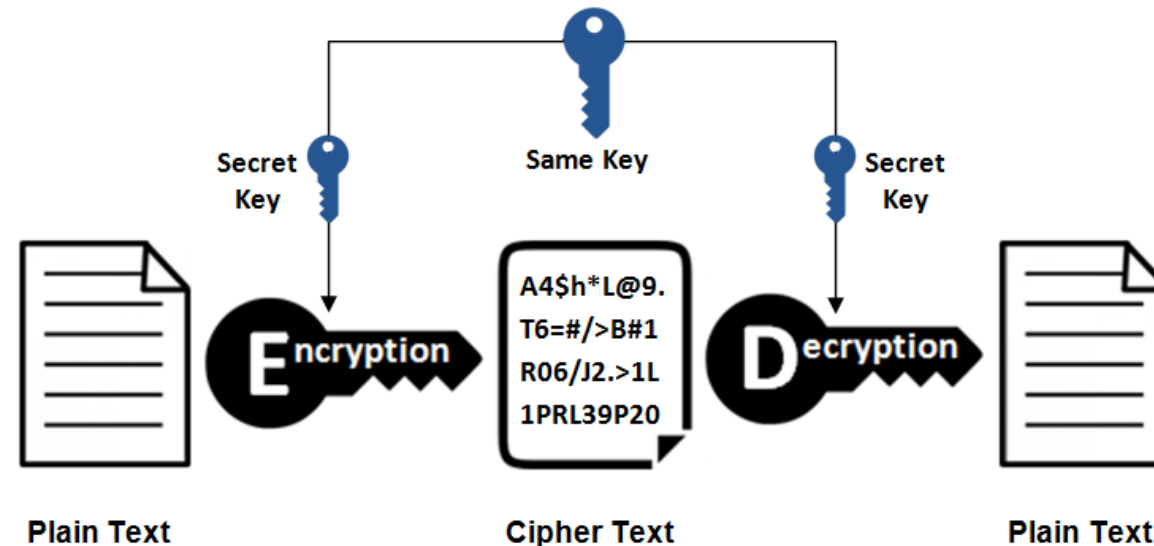
Confidentiality Design

- Disclosure protection can be achieved in several ways using cryptographic and masking techniques.
 - **Masking** is useful for disclosure protection when data is displayed on the screen or on printed forms
 - **Cryptographic techniques** are useful for assurance of confidentiality when the data is transmitted or stored in transactional data stores or offline archives



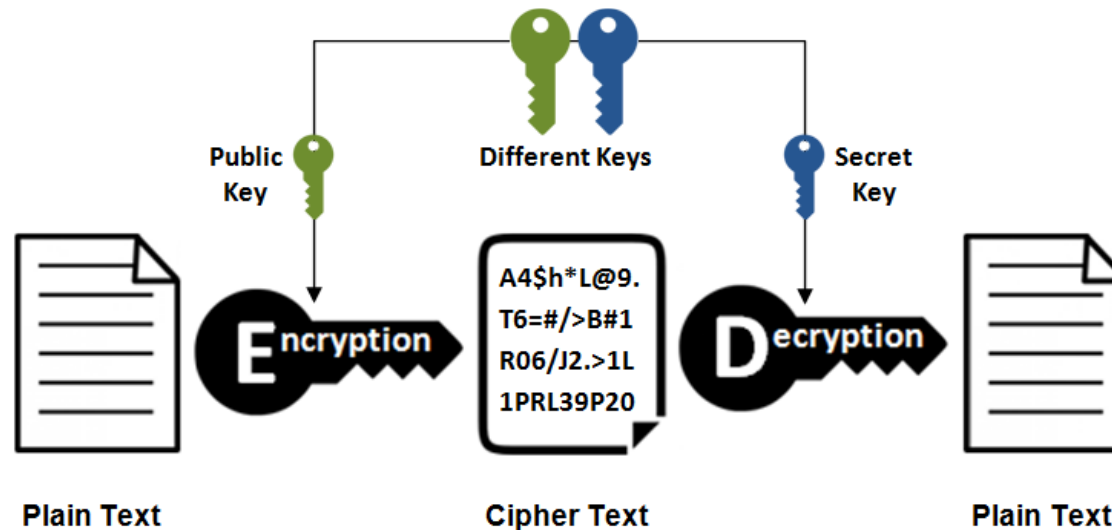
Symmetric Algorithms

- **Symmetric algorithms** are characterized by using a single key for encryption and decryption operations that is shared between the sender and the receiver
 - E.g., DES, 3DES, Blowfish, AES



Asymmetric Algorithms

- Two keys that are mathematically related to each other are used
 - **Private key**: the key to be held secret
 - **Public key**: disclosed to anyone with whom secure communications and transactions need to occur.
 - It should be computationally infeasible to derive the private key from the public key.



Comparison

Symmetric Algorithms

- Key exchange and management
 - Out-of-band secure mechanism to exchange the key
 - Number of keys among N persons: 1
- Efficiency
 - Much faster
- Non-repudiation not addressed

Asymmetric Algorithms

- Key exchange and management
 - Based on public key infrastructure (PKI)
 - Number of keys among N persons: N
- Efficiency
 - Much slower
- Non-repudiation addressed

Question time!

- A developer wants to develop a mobile application
 - He is now designing the signup feature
 - Should he send the username / password to the backend server during this process?
- He worries about if SSL cert pinning is not done correctly, and hacker may use sniffer to steal the username and password at the very beginning
 - Can you propose a good design to let he feel safer?



Digital Certificate

- Digital certificate standard is ITU X.509 version 3 (X.509 v3)

X.509 v3 Certificate			
Required	Certificate	Version	
		Algorithm Identifier	
		Serial Number (unique identifier for each certificate the CA issues)	
	Issuer	Distinguished CA name	
	Validity Period	Period from and to which the certificate will be valid	
	Subject	Name (same as the Issuer for a root certificate)	
		Public key	Algorithm Identifier
Value			
Optional	Unique Identifier	Issuer	
		Subject	
	Extensions	Additional certificate and policy information	
Digital Signature of the CA			

Certificate:

Data:

Version: 3 (0x2)

Serial Number:

03:04:54:08:f9:ff:10:92:e1:69:fe:49:8f:78:d3:6d:dc:47

Signature Algorithm: sha256WithRSAEncryption

Issuer: C = US, O = Let's Encrypt, CN = R3

Validity

Not Before: Jul 15 08:01:49 2021 GMT

Not After : Oct 13 08:01:48 2021 GMT

Subject: CN = *.wikipedia.org

Subject Public Key Info:

Public Key Algorithm: id-ecPublicKey

Public-Key: (256 bit)

pub:

04:a5:9a:47:b2:d3:fc:a7:df:de:f6:cb:45:62:0a:

d3:c1:a7:38:de:20:bd:d7:10:7d:58:73:de:8d:a1:

99:70:0c:dd:ab:91:3f:0e:83:97:1b:4f:a2:99:f3:

f8:30:73:ef:da:be:91:25:18:7a:d6:da:bf:e5:e9:

72:a3:41:31:7a

ASN1 OID: prime256v1

NIST CURVE: P-256

X509v3 extensions:

X509v3 Key Usage: critical

Digital Signature

Integrity Design

- Integrity of software and data can be accomplished using any one of the following techniques or a combination of the techniques
 - Hashing (or hash functions)
 - Referential integrity design
 - Resource locking

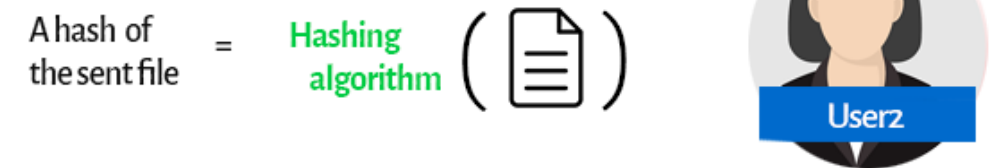
Hashing (Hash Functions)

- Hash functions are used to condense variable length inputs into an irreversible, collision free, fixed-sized output known as a message digest or hash value.
 - MD5, SHA-1, SHA256
- Hashing can also be used for confidentiality design
 - E.g., password storage on Web server

1. Step 1: User1 sends a file to User2 alongside its checksum



2. User2 receives the file and uses the same hashing algorithm



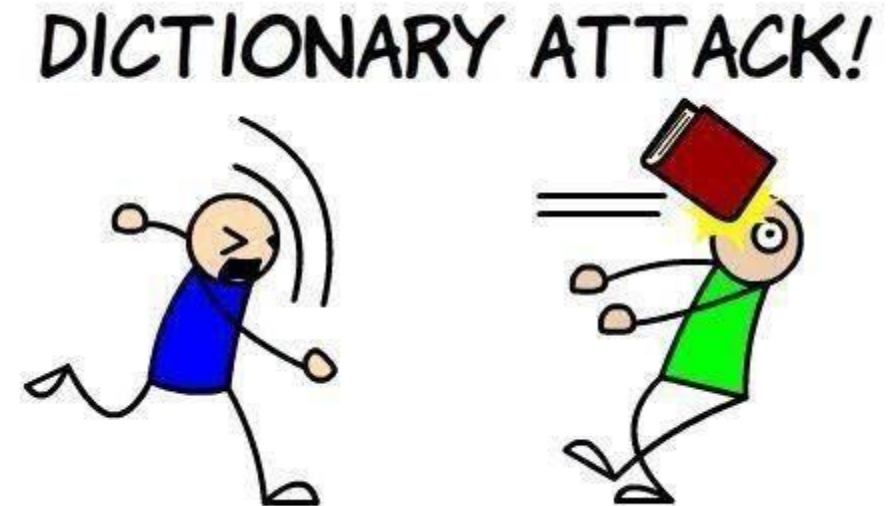
3. User 2 compares both hashes. If they are the same, the file is the same as well



Prebuilt dictionary attacks

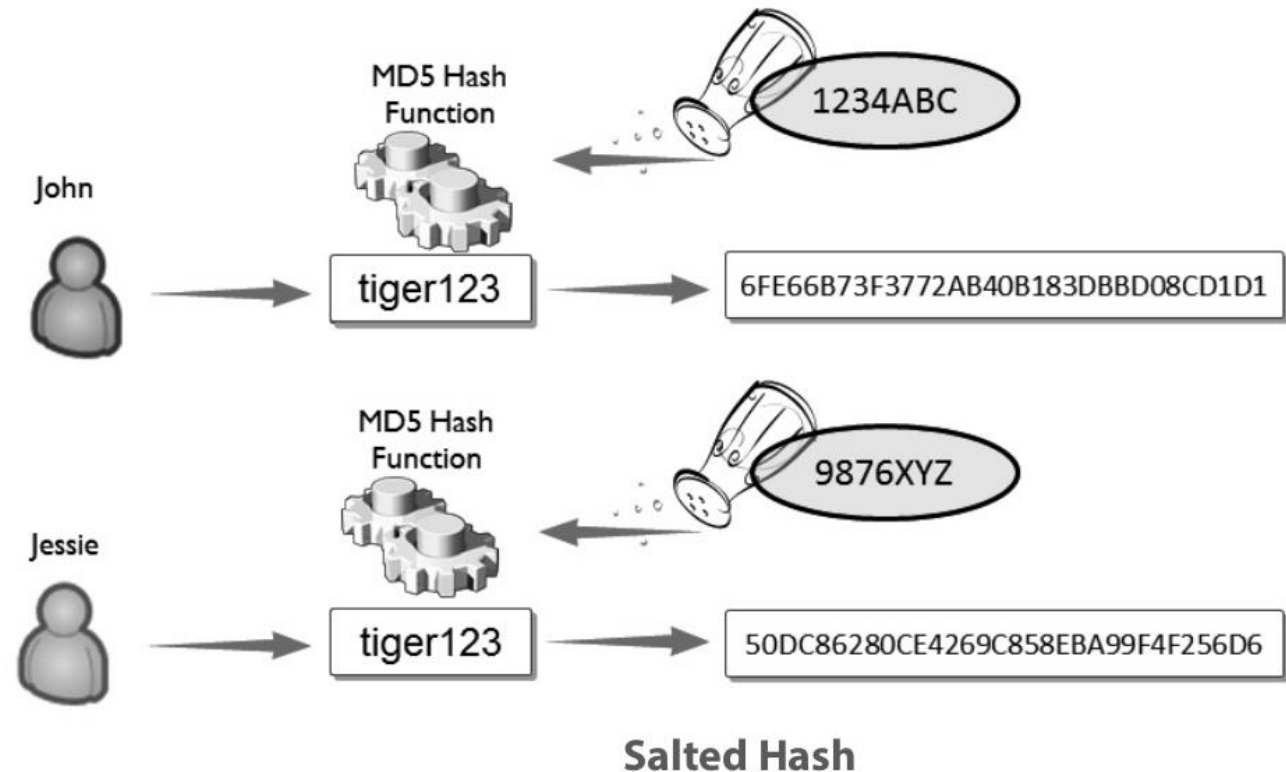
- A dictionary attack is an attempt to thwart security protection mechanisms by using an exhaustive list (like a list of words from a dictionary)

```
root@192: /
File Edit View Search Terminal Help
[ATTEMPT] target testasp.vulnweb.com - login "admin" - pass "pokemon" - 174 of 14344400 [child 14] (0/0)
[ATTEMPT] target testasp.vulnweb.com - login "admin" - pass "maggie" - 175 of 14344400 [child 11] (0/0)
[ATTEMPT] target testasp.vulnweb.com - login "admin" - pass "corazon" - 176 of 14344400 [child 1] (0/0)
[ATTEMPT] target testasp.vulnweb.com - login "admin" - pass "chicken" - 177 of 14344400 [child 7] (0/0)
[ATTEMPT] target testasp.vulnweb.com - login "admin" - pass "pepper" - 178 of 14344400 [child 3] (0/0)
[ATTEMPT] target testasp.vulnweb.com - login "admin" - pass "cristina" - 179 of 14344400 [child 5] (0/0)
[ATTEMPT] target testasp.vulnweb.com - login "admin" - pass "rainbow" - 180 of 14344400 [child 6] (0/0)
[ATTEMPT] target testasp.vulnweb.com - login "admin" - pass "kisses" - 181 of 14344400 [child 2] (0/0)
[ATTEMPT] target testasp.vulnweb.com - login "admin" - pass "manuel" - 182 of 14344400 [child 4] (0/0)
[ATTEMPT] target testasp.vulnweb.com - login "admin" - pass "myspace" - 183 of 14344400 [child 9] (0/0)
[ATTEMPT] target testasp.vulnweb.com - login "admin" - pass "rebelde" - 184 of 14344400 [child 15] (0/0)
[ATTEMPT] target testasp.vulnweb.com - login "admin" - pass "angeli" - 185 of 14344400 [child 10] (0/0)
[ATTEMPT] target testasp.vulnweb.com - login "admin" - pass "ricardo" - 186 of 14344400 [child 12] (0/0)
[ATTEMPT] target testasp.vulnweb.com - login "admin" - pass "babygurl" - 187 of 14344400 [child 13] (0/0)
[ATTEMPT] target testasp.vulnweb.com - login "admin" - pass "heaven" - 188 of 14344400 [child 0] (0/0)
[ATTEMPT] target testasp.vulnweb.com - login "admin" - pass "55555" - 189 of 14344400 [child 8] (0/0)
[ATTEMPT] target testasp.vulnweb.com - login "admin" - pass "baseball" - 190 of 14344400 [child 14] (0/0)
[ATTEMPT] target testasp.vulnweb.com - login "admin" - pass "martin" - 191 of 14344400 [child 7] (0/0)
[ATTEMPT] target testasp.vulnweb.com - login "admin" - pass "greenday" - 192 of 14344400 [child 3] (0/0)
[ATTEMPT] target testasp.vulnweb.com - login "admin" - pass "november" - 193 of 14344400 [child 5] (0/0)
[ATTEMPT] target testasp.vulnweb.com - login "admin" - pass "alyssa" - 194 of 14344400 [child 11] (0/0)
[ATTEMPT] target testasp.vulnweb.com - login "admin" - pass "madison" - 195 of 14344400 [child 6] (0/0)
[ATTEMPT] target testasp.vulnweb.com - login "admin" - pass "mother" - 196 of 14344400 [child 4] (0/0)
[ATTEMPT] target testasp.vulnweb.com - login "admin" - pass "123321" - 197 of 14344400 [child 2] (0/0)
```



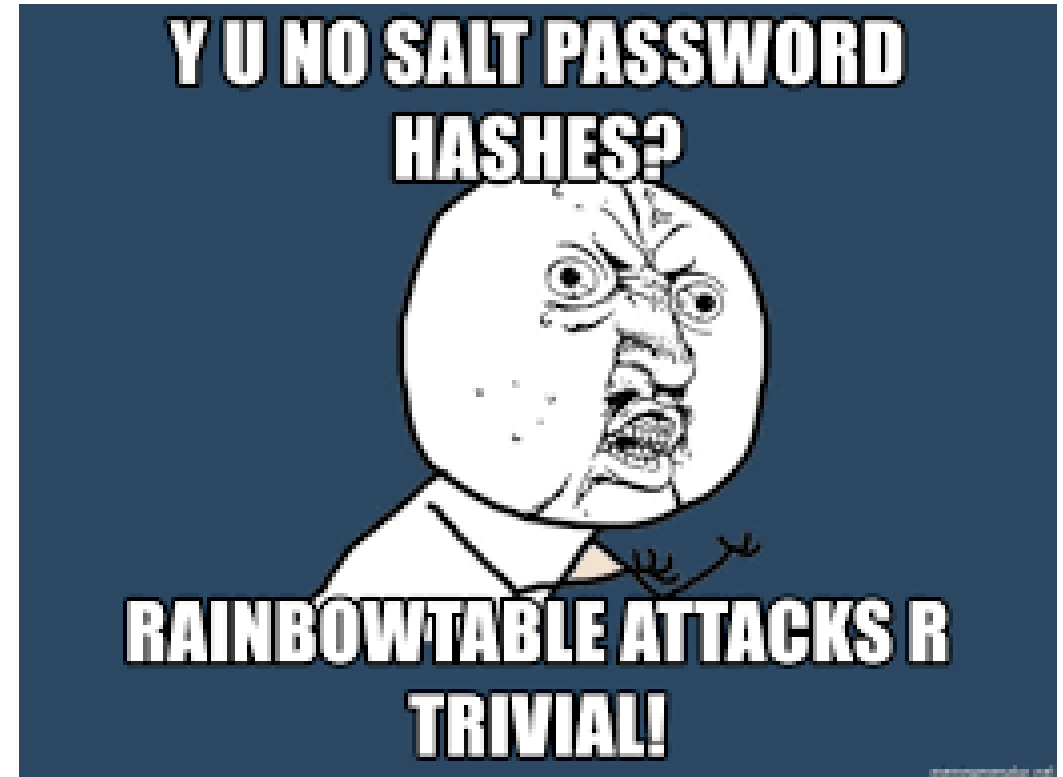
Salted hash

- Design considerations should take into account the security aspects related to the generation of the salt, which should be unique to each user and random



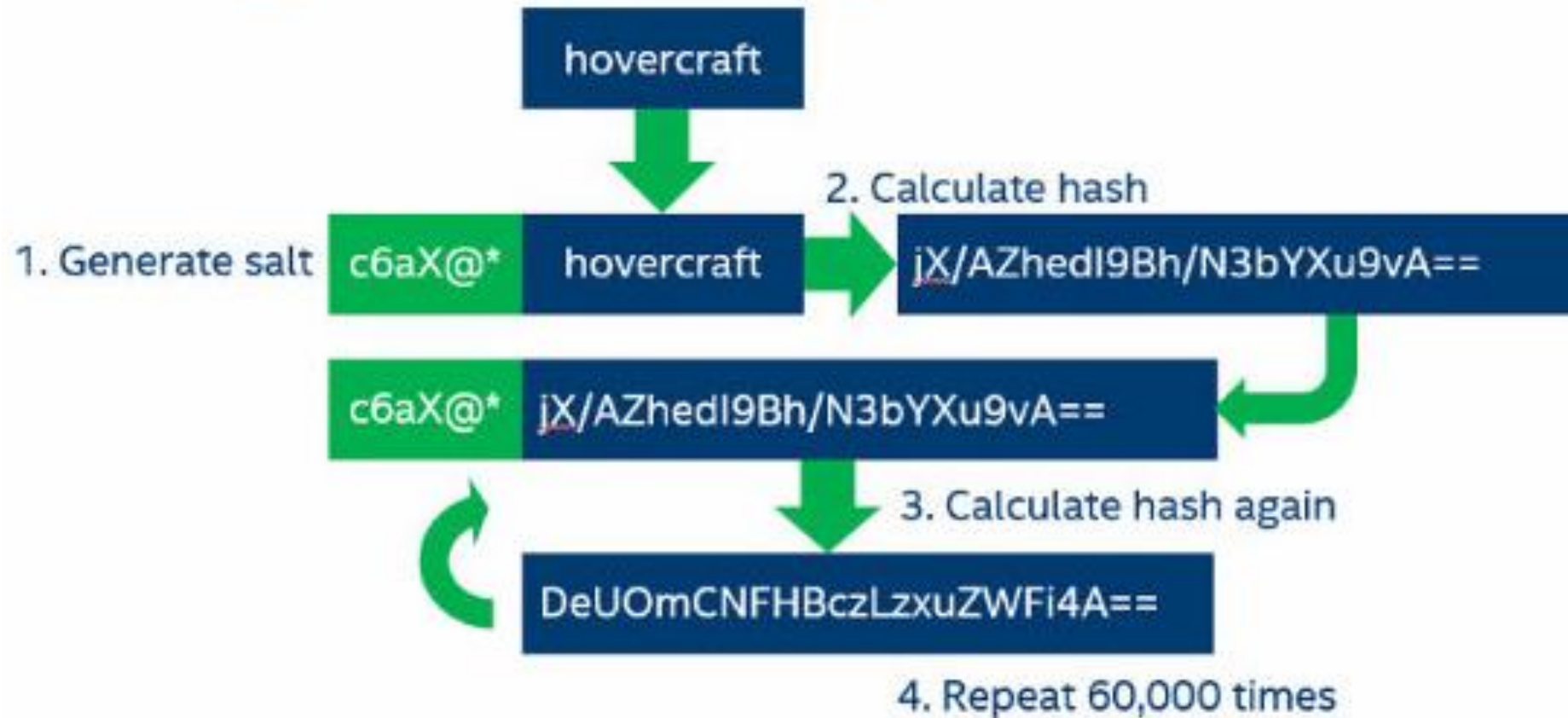
Question time

- How are you creating the salt for password?
 - All password use the same salt?
 - When/where/how you are creating the salt?



Example...

Strategy for Strong Hashing



Referential Integrity

- Referential integrity is a property of data stating references within it are valid.
- Referential integrity design requires that, in the context of relational databases, *every value of one attribute (column) of a relation (table) to exist as a value of another attribute (column) in a different (or the same) relation (table).*

CUSTOMER

Customer_ID	First_Name	Last_Name	User_Name	Password
1	John	Smith	Jsmith1	9E107D9D372BB682
2	John	Smith	Jsmith2	6A207E9F154CC753
3	Mary	Johnson	Mjohnson	8A219E7E135AA717

ORDER

Order_ID	Customer_ID	Order_Amount	Order_Date
101	1	129.00	January 20, 2009
102	3	289.00	March 06, 2009
103	1	129.00	August 15, 2009

ORDER_DETAIL

Order_Detail_ID	Order_ID	Product_Code	Product_Desc
1001	101	CSSLPEX1	CSSLP Self Assessment Exam 1
1002	102	CSSLPEX1	CSSLP Self Assessment Exam 1
1003	102	CSSLPEX2	CSSLP Self Assessment Exam 2
1004	102	CSSLPEX3	CSSLP Self Assessment Exam 3
1005	103	CSSLPEX1	CSSLP Self Assessment Exam 1

Tables without Referential integrity design

CUSTOMER

Customer_ID	First_Name	Last_Name	User_Name	Password
1	John	Smith	Jsmith1	9E107D9D372BB682

ORDER

Order_ID	Order_Amount	Order_Date
101	129.00	January 20, 2009

ORDER_DETAIL

Order_Detail_ID	Product_Code	Product_Desc
1001	CSSLPEX1	CSSLP Self Assessment Exam 1

Tables without Referential integrity design

CUSTOMER

Customer_ID	First_Name	Last_Name	User_Name	Password
1	John	Smith	Jsmith1	9E107D9D372BB682
2	John	Smith	Jsmith2	6A207E9F154CC753

ORDER

Order_ID	Order_Amount	Order_Date
101	129.00	January 20, 2009

ORDER_DETAIL

Order_Detail_ID	Product_Code	Product_Desc
1001	CSSLPEX1	CSSLP Self Assessment Exam 1

Tables without Referential integrity design

CUSTOMER

Customer_ID	First_Name	Last_Name	User_Name	Password
1	John	Smith	Jsmith1	9E107D9D372BB682
2	John	Smith	Jsmith2	6A207E9F154CC753
3	Mary	Johnson	Mjohson	8A219E7E135AA717

ORDER

Order_ID	Order_Amount	Order_Date
101	129.00	January 20, 2009
102	289.00	March 06, 2009

ORDER_DETAIL

Order_Detail_ID	Product_Code	Product_Desc
1001	CSSLPEX1	CSSLP Self Assessment Exam 1
1002	CSSLPEX1	CSSLP Self Assessment Exam 1
1003	CSSLPEX2	CSSLP Self Assessment Exam 2
1004	CSSLPEX3	CSSLP Self Assessment Exam 3

Tables without Referential integrity design

CUSTOMER

Customer_ID	First_Name	Last_Name	User_Name	Password
1	John	Smith	Jsmith1	9E107D9D372BB682
2	John	Smith	Jsmith2	6A207E9F154CC753
3	Mary	Johnson	Mjohson	8A219E7E135AA717

ORDER

Order_ID	Order_Amount	Order_Date
101	129.00	January 20, 2009
102	289.00	March 06, 2009
103	129.00	August 15, 2009

ORDER_DETAIL

Order_Detail_ID	Product_Code	Product_Desc
1001	CSSLPEX1	CSSLP Self Assessment Exam 1
1002	CSSLPEX1	CSSLP Self Assessment Exam 1
1003	CSSLPEX2	CSSLP Self Assessment Exam 2
1004	CSSLPEX3	CSSLP Self Assessment Exam 3
1005	CSSLPEX1	CSSLP Self Assessment Exam 1

Tables without Referential integrity design

CUSTOMER

Customer_ID	First_Name	Last_Name	User_Name	Password
1	John	Smith	Jsmith1	9E107D9D372BB682
2	John	Smith	Jsmith2	6A207E9F154CC753
3	Marv	Johnson	Mjohson	8A219E7E135AA717

ORDER

Order_ID	Order_Amount	Order_Date
101	129.00	January 20, 2009
102	289.00	March 06, 2009
103	129.00	August 15, 2009

orphan

ORDER_DETAIL

Order_Detail_ID	Product_Code	Product_Desc
1001	CSSLPEX1	CSSLP Self Assessment Exam 1
1002	CSSLPEX1	CSSLP Self Assessment Exam 1
1003	CSSLPEX2	CSSLP Self Assessment Exam 2
1004	CSSLPEX3	CSSLP Self Assessment Exam 3
1005	CSSLPEX1	CSSLP Self Assessment Exam 1

orphan

Tables with Referential integrity design

CUSTOMER

Customer_ID	First_Name	Last_Name	User_Name	Password
1	John	Smith	Jsmith1	9E107D9D372BB682
2	John	Smith	Jsmith2	6A207E9F154CC753
3	Mary	Johnson	Mjohson	8A219E7E135AA717

ORDER

Order_ID	Customer_ID	Order_Amount	Order_Date
101	1	129.00	January 20, 2009
102	3	289.00	March 06, 2009
103	1	129.00	August 15, 2009

ORDER_DETAIL

Order_Detail_ID	Order_ID	Product_Code	Product_Desc
1001	101	CSSLPEX1	CSSLP Self Assessment Exam 1
1002	102	CSSLPEX1	CSSLP Self Assessment Exam 1
1003	102	CSSLPEX2	CSSLP Self Assessment Exam 2
1004	102	CSSLPEX3	CSSLP Self Assessment Exam 3
1005	103	CSSLPEX1	CSSLP Self Assessment Exam 1

Referential integrity design requires that, in the context of relational databases, every value of one attribute (column) of a relation (table) to exist as a value of another attribute (column) in a different (or the same) relation (table).

Or, simply, use **primary keys** and related **foreign keys** in the database to assure data integrity

Do your application
allow delete
customer?
What should you do?

Resource locking

- Two concurrent operations are not allowed on the same object
 - E.g., say a record in the database
- An online banking app with web client and mobile client
 - Can they login at the same time?
 - If pressing transfer button at the same time?

Availability Design

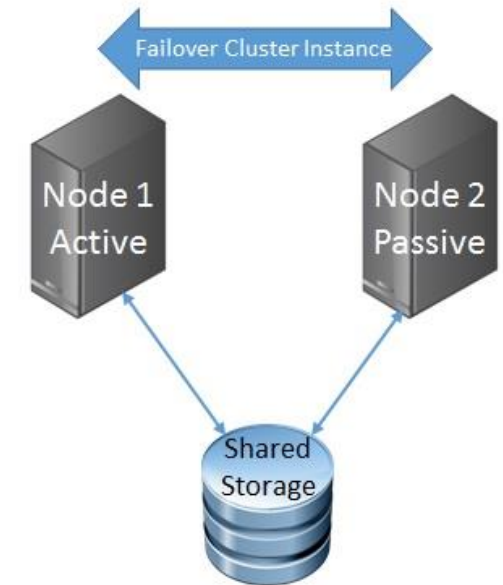
Replication, Failover and Scalability techniques can also be used to design the software for availability

Replication

- A single point of failure is characterized by having no redundancy capabilities and this can undesirably affect end-users when a failure occurs
- By replicating data, databases and software across multiple computer systems, a degree of redundancy is made possible
- Replication usually follows a master-slave or primary-secondary backup scheme

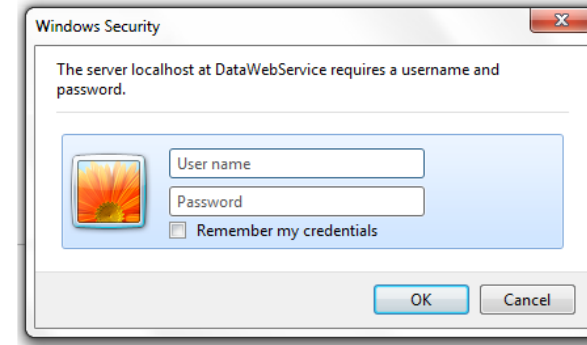
Availability Design

- Failover
 - Failover refers to the automatic switching from an active transactional software, server, system, hardware component or network to a standby (or redundant) system.
- Scalability Techniques
 - Vertical scaling means that additional resources are added to the existing node
 - Memory, storage, etc.
 - Horizontal scaling means that newer nodes are added to the existing nodes



Authentication Design

- Basic authentication
 - What (only) he knows (e.g., password)
 - What he is (e.g., biometric)
 - What he has (e.g., access token/smartphone)
- Consider multi-factor authentication and Single Sign-On



Log in with Facebook



Sign in with Google

Sign In

[Create an Account](#)

Email or Nickname

Password

[Forgot your password?](#)

Submit 

Authorization Design

- **Role-based access control (RBAC)** is predominantly the way authorization decisions are made in most applications.
- **User-based access control (UBAC)** is another way if the application is for very limited number of users and various type of permissions.
- When designing for authorization, give special attention to its impact on performance, and to the principles of separation of duties and least privilege.

Question time

- Some of your team has (Super) Admin account.
- Admin forgot password; will you handle?
- How do you create that admin account?

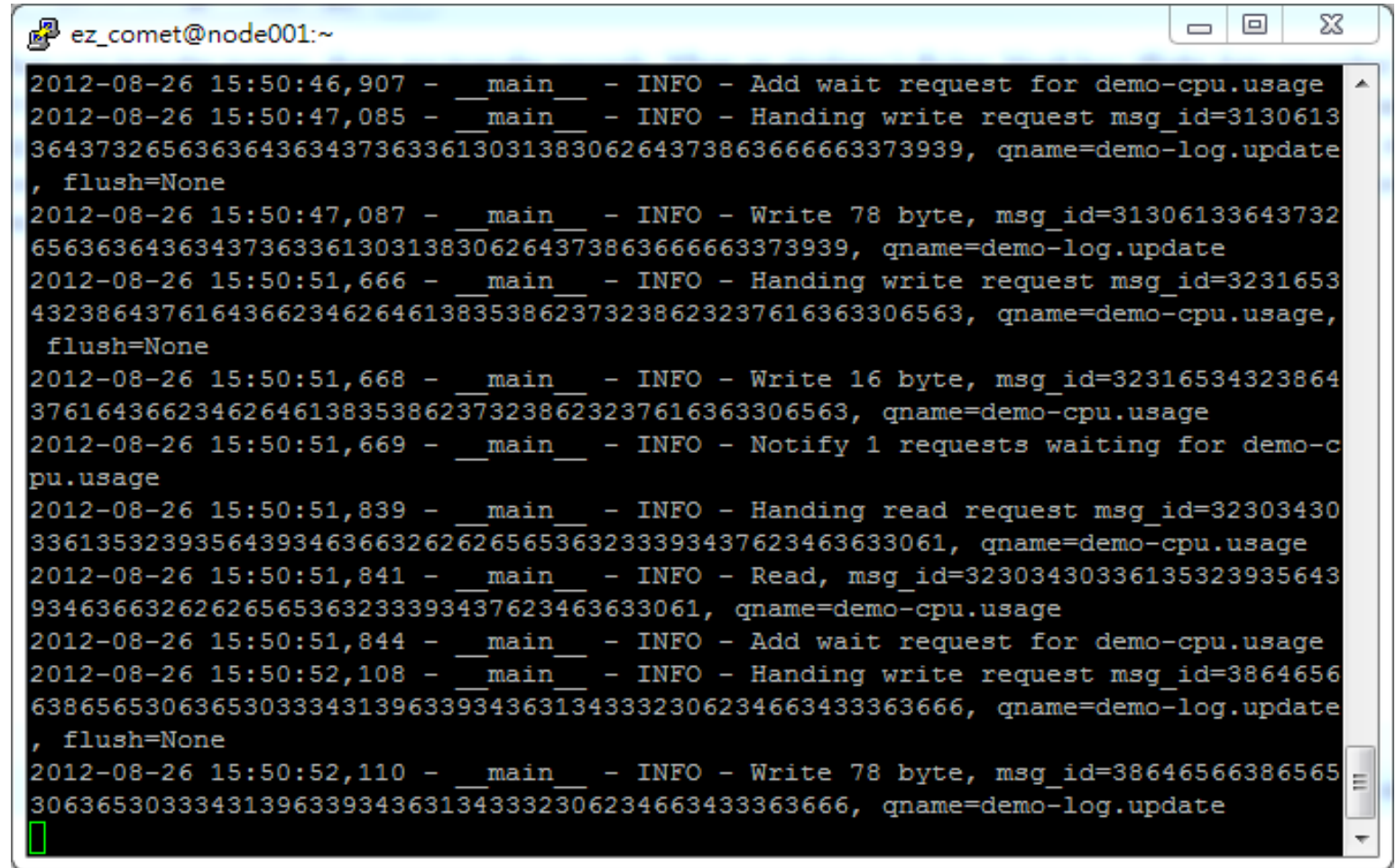


Accountability Design

- Design for auditing has been proven to be extremely important in the event of a breach, primarily for forensic purposes, and so it should be factored into the design of the software from the very beginning.
- Log data should include the **'who', 'what', 'where', and 'when'** aspects of software operations.
- Sensitive data should never be logged in plaintext form

Accountability Design

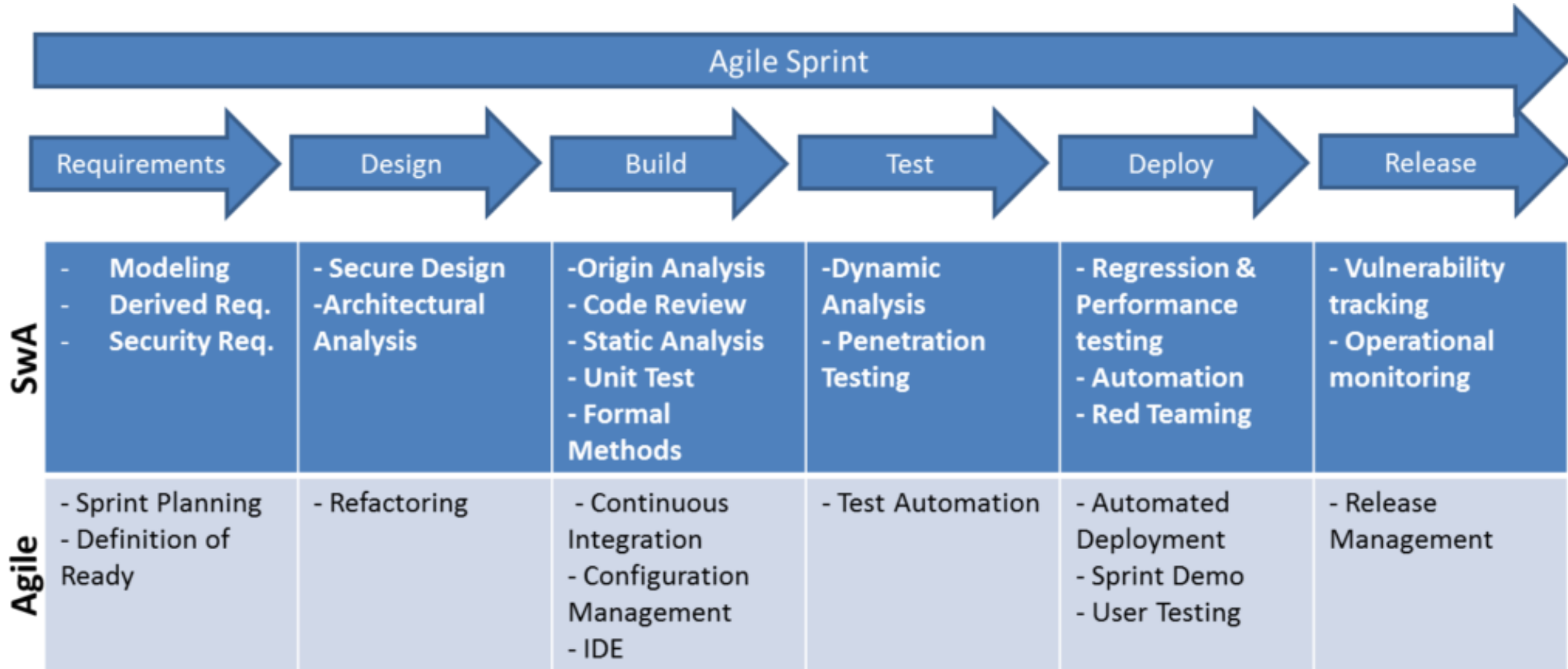
- Good logging design
 - Easy for developer to debug
 - Does NOT contain ANY user information
 - Only store limited information
 - Log erase / rotation

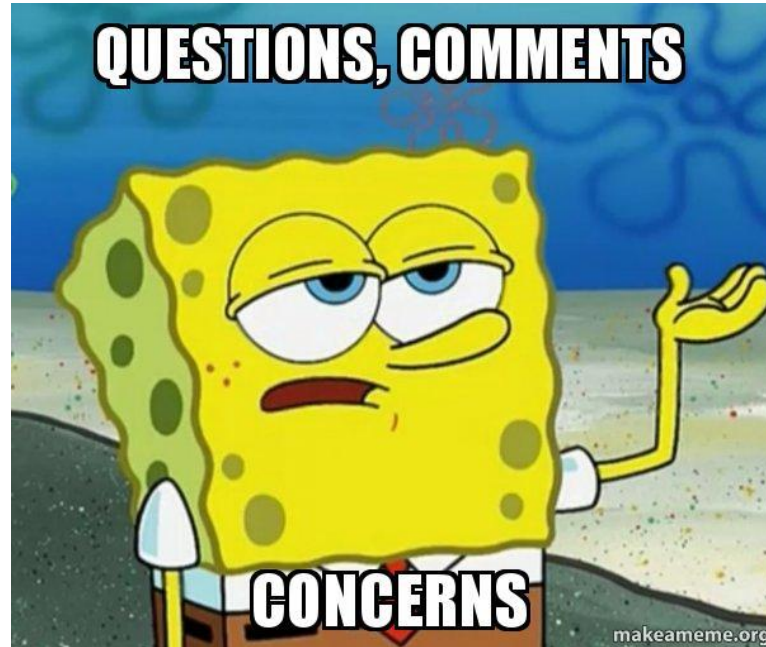


```
ez_comet@node001:~  
2012-08-26 15:50:46,907 - __main__ - INFO - Add wait request for demo-cpu.usage  
2012-08-26 15:50:47,085 - __main__ - INFO - Handling write request msg_id=3130613  
364373265636364363437363361303138306264373863666663373939, qname=demo-log.update  
, flush=None  
2012-08-26 15:50:47,087 - __main__ - INFO - Write 78 byte, msg_id=31306133643732  
65636364363437363361303138306264373863666663373939, qname=demo-log.update  
2012-08-26 15:50:51,666 - __main__ - INFO - Handling write request msg_id=3231653  
432386437616436623462646138353862373238623237616363306563, qname=demo-cpu.usage,  
flush=None  
2012-08-26 15:50:51,668 - __main__ - INFO - Write 16 byte, msg_id=32316534323864  
37616436623462646138353862373238623237616363306563, qname=demo-cpu.usage  
2012-08-26 15:50:51,669 - __main__ - INFO - Notify 1 requests waiting for demo-c  
pu.usage  
2012-08-26 15:50:51,839 - __main__ - INFO - Handling read request msg_id=32303430  
33613532393564393463663262626565363233393437623463633061, qname=demo-cpu.usage  
2012-08-26 15:50:51,841 - __main__ - INFO - Read, msg_id=32303430336135323935643  
93463663262626565363233393437623463633061, qname=demo-cpu.usage  
2012-08-26 15:50:51,844 - __main__ - INFO - Add wait request for demo-cpu.usage  
2012-08-26 15:50:52,108 - __main__ - INFO - Handling write request msg_id=3864656  
638656530636530333431396339343631343332306234663433363666, qname=demo-log.update  
, flush=None  
2012-08-26 15:50:52,110 - __main__ - INFO - Write 78 byte, msg_id=38646566386565  
30636530333431396339343631343332306234663433363666, qname=demo-log.update  
█
```

SECURE AGILE DEVELOPMENT

Secure Agile Development





QUESTION AND COMMENTS?

But I have question

Remember in pervious lecture, I asked the question:

“How to store API credentials in your application / Dockerfile”

At the design level, how to have a secure design?

Confidentiality Design

- The credentials stored in source code -> zero marks
- How about a configuration file?
- Download the configuration file during the 1st launch?
- How to store that file in a secure location?
- Encrypt that file?

Integrity Design

- All users who downloaded that application share the same database credentials?
- If you allow that database credentials have CHANGE password permission, what will happen?
- If you allow that database credentials have GRANT / UPDATE / DELETE permission, what will happen?
- The database credentials permission?
- How about each user has different credentials?

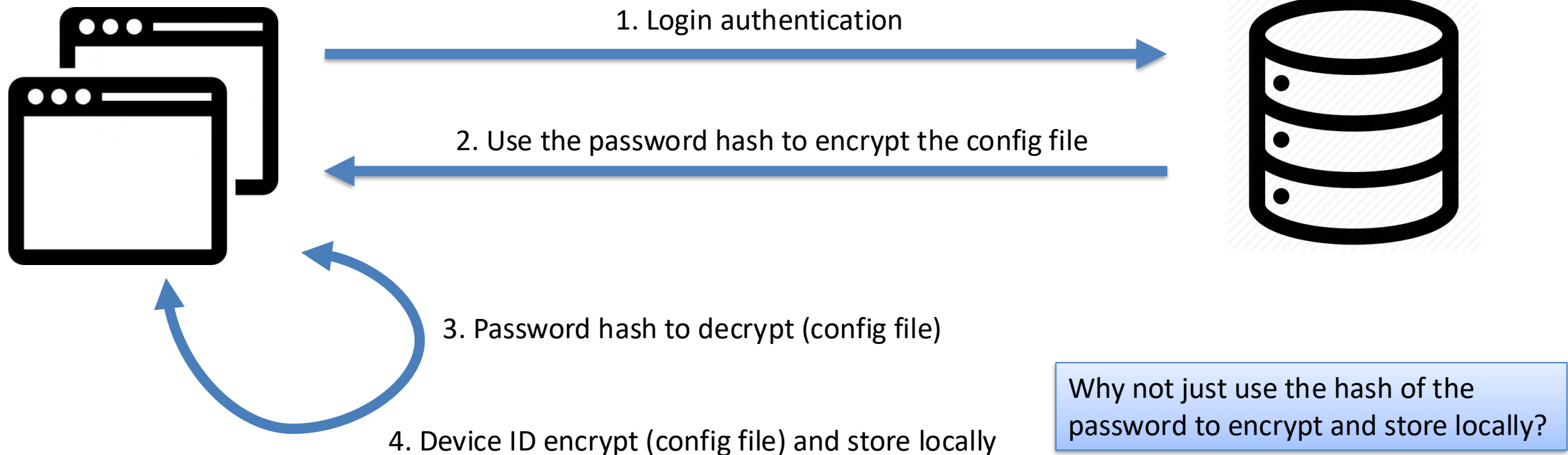
- Availability Design
 - DDoS the database directly?
- Authentication Design
 - How can I ensure the right user can access the database?
- Authorization Design
 - Permission...again...
- Accountability Design
 - LOG in both sides?

NOT a suggested solution

Can work without HTTPS?

Is it true?

Can you think of other attack?



Another exercise

Username	Password
Raymond	123456
Wing Keong	abcdef
Tram	987654321
Marcus	qwerty

What is the problem?
CIAAAA?

Is it better?

Username	MD5
Raymond	e10adc3949ba59abbe56e057f20f883e
Wing Keong	e80b5017098950fc58aad83c8c14978e
Tram	6ebe76c9fb411be97b3b0d48b791a7c9
Marcus	d8578edf8458ce06fbc5bb76a58c5ca4

OK...how about now?

Username	SHA256
Raymond	8d969eef6ecad3c29a3a629280e686cf0c3f5d5a86aff3ca12020c923adc6c92
Wing Keong	bef57ec7f53a6d40beb640a780a639c83bc29ac8a9816f1fc6c5c6dcd93c4721
Tram	8a9bcf1e51e812d0af8465a8dbcc9f741064bf0af3b3d08e6b0246437c19f7fb
Marcus	65e84be33532fb784c48129675f9eff3a682b27168c0ea744b2cf58ee02337c5

Or like this?

Username	SHA256	Salt
Raymond	b117bd415f0348b8dfb542423c62300553c18b90eaeaf4df515e50ad4432026d	QxLUF1bgIAdeQX
Wing Keong	c48093877d5d058ccbff356d10f21436ea216fa5d895836615c1a19ce511ba5a	bv5PehSMfV11Cd
Tram	408c65e20a858300faf3fb5d49f3d3f3175d3cf6c4a386e8351d84baaf452e04	YYLmfY6lehjZMQ
Marcus	eb0a7bf8cff25a1f7ed706079ddd3ffcfabaa3a4219c6e5e0fc0febd3dd78c98	KjaweUIYeKSWDI

Add more...

Username	SHA256	Salt	Account Create date	Last Login datetime	Sessions	Fail login count	Password History
Raymond	b117bd415f0348b8dfb542423c62300553c18b90eaeaf4df515e50ad4432026d	QxLUF1bgIAdeQX					
Wing Keong	c48093877d5d058ccbff356d10f21436ea216fa5d895836615c1a19ce511ba5a	bv5PehSMfV11Cd					
Tram	408c65e20a858300faf3fb5d49f3d3f3175d3cf6c4a386e8351d84baaf452e04	YYLmfY6lehjZMQ					
Marcus	eb0a7bf8cff25a1f7ed706079ddd3ffcfaaa3a4219c6e5e0fc0febd3dd78c98	KjaweUIYeKSWDI					

Passwordless?

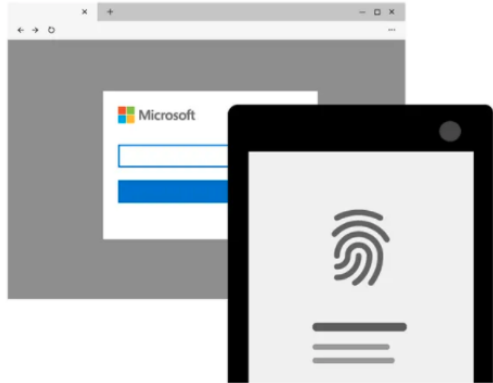
Microsoft accounts can now go fully passwordless 36

You can delete your Microsoft account password

By [Tom Warren](#) | [@tomwarren](#) | Sep 15, 2021, 9:00am EDT

If you buy something from a Verge link, Vox Media may earn a commission. See our [ethics statement](#).

[f](#) [t](#) [SHARE](#)





Subscribe to get the best Verge-approved tech deals of the week.

Email (required)

By signing up, you agree to our [Privacy Notice](#) and European users agree to the data transfer policy.

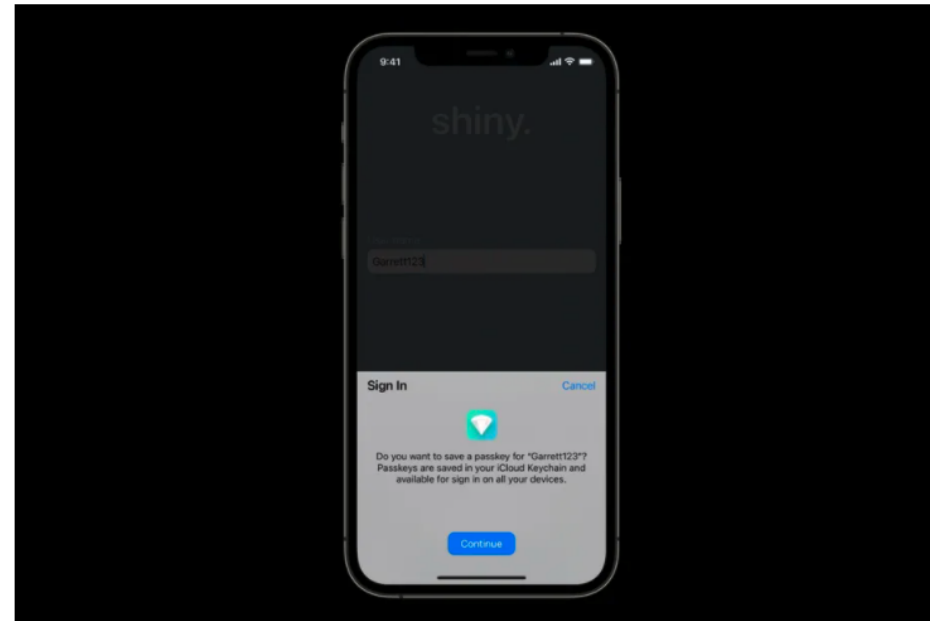
SUBSCRIBE

iOS 15 and macOS 12 take a small but significant step towards a password-less future 20

A more secure approach than traditional passwords

By [Jon Porter](#) | [@JonPorty](#) | Jun 11, 2021, 7:32am EDT

[f](#) [t](#) [SHARE](#)



In the future you'll be able to sign up for a service without needing to create a password at all. | Image: Apple



Subscribe to get the best Verge-approved tech deals of the week.

Email (required)

By signing up, you agree to our [Privacy Notice](#) and European users agree to the data transfer policy.

SUBSCRIBE

In fact, WhatsApp is the first one...

What is my WhatsApp password?

 8 Answers



Shriraj Hegde, lives in Mumbai, Maharashtra, India

Answered 3 years ago

There is no password to whatsapp, but whatsapp uses an OTP during verification which is required only at the time of number verification

15.8K views · View upvotes



3



Related Questions

[More Answers Below](#)

[How can I get my WhatsApp password?](#)

[What if you can't remember your WhatsApp password?](#)

[How do I retrieve my WhatsApp username and password?](#)

[What is my WhatsApp username and password?](#)

[How do I get my WhatsApp ID and password from my existing account?](#)



Rajesh Tangudu, lives in Hyderabad, Telangana, India

Answered 3 years ago

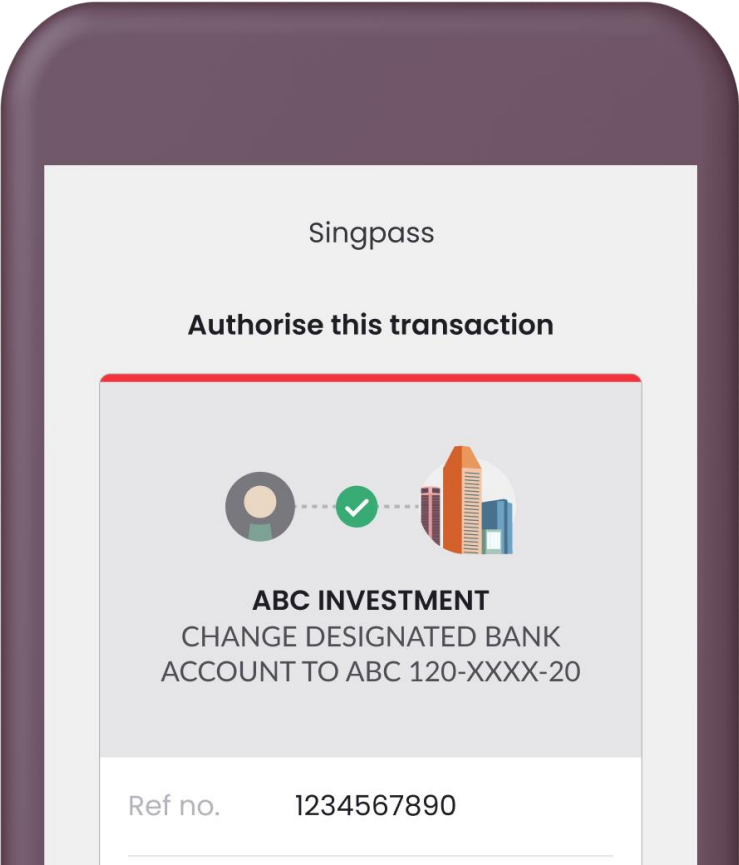
I believe there will be no password option in Whatsapp. You can directly call from the registered number to get the OTP(One Time Password) or you will get the OTP message to your registered number at the time of installation & setup.

4.4K views · View upvotes

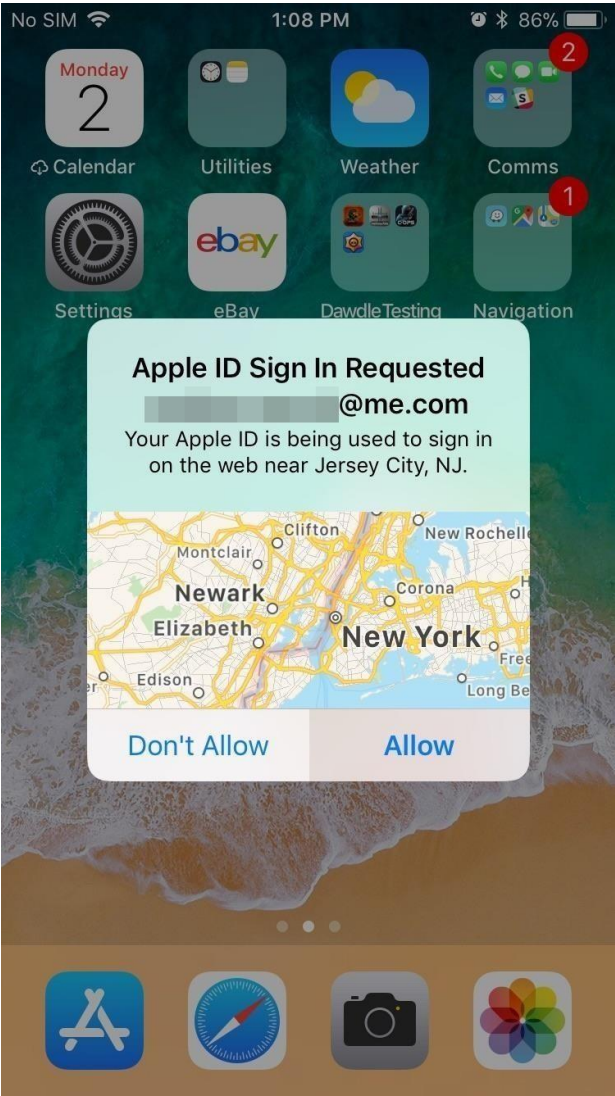
Design of a passwordless system?

- Need an Authenticator App / devices you have. (Something you have)
- The initial process will be sending an SMS / Email to validate you when setting up the Authenticator App / devices.
- Subsequently, the login will need to approve by the Authenticator App / devices.

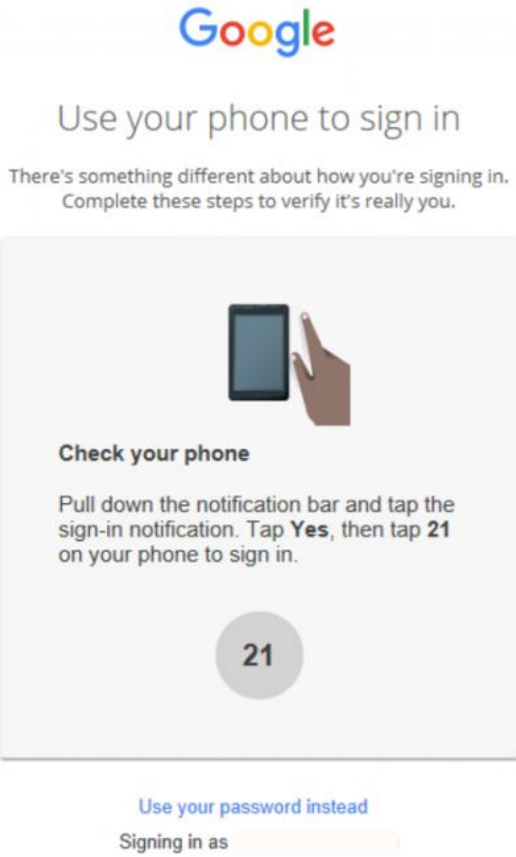
Some examples

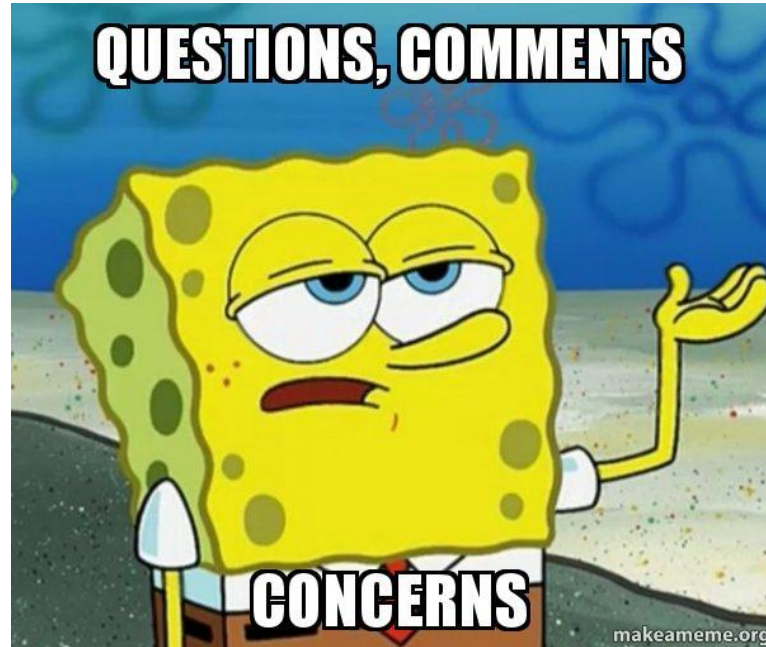


Scan an QR Code



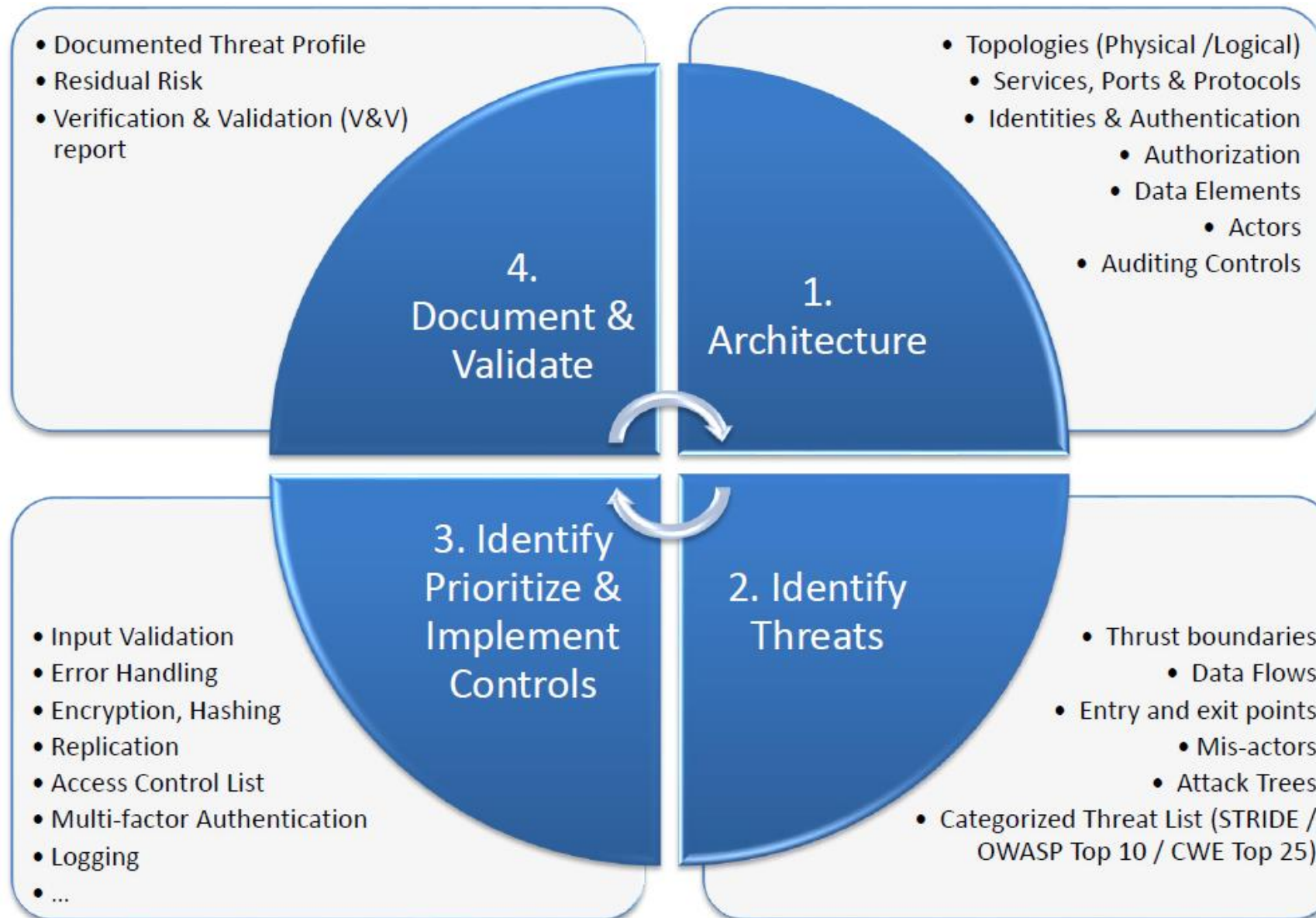
Approve from devices you already sign in





QUESTION AND COMMENTS?

Design Process (Threat Modeling Process)



Secure Design Architecture



Identify physical topology



Identify logical topology



Identify Human and Non-Human Actors of the system



Identify Data Elements



Generate a Data Access Control Matrix



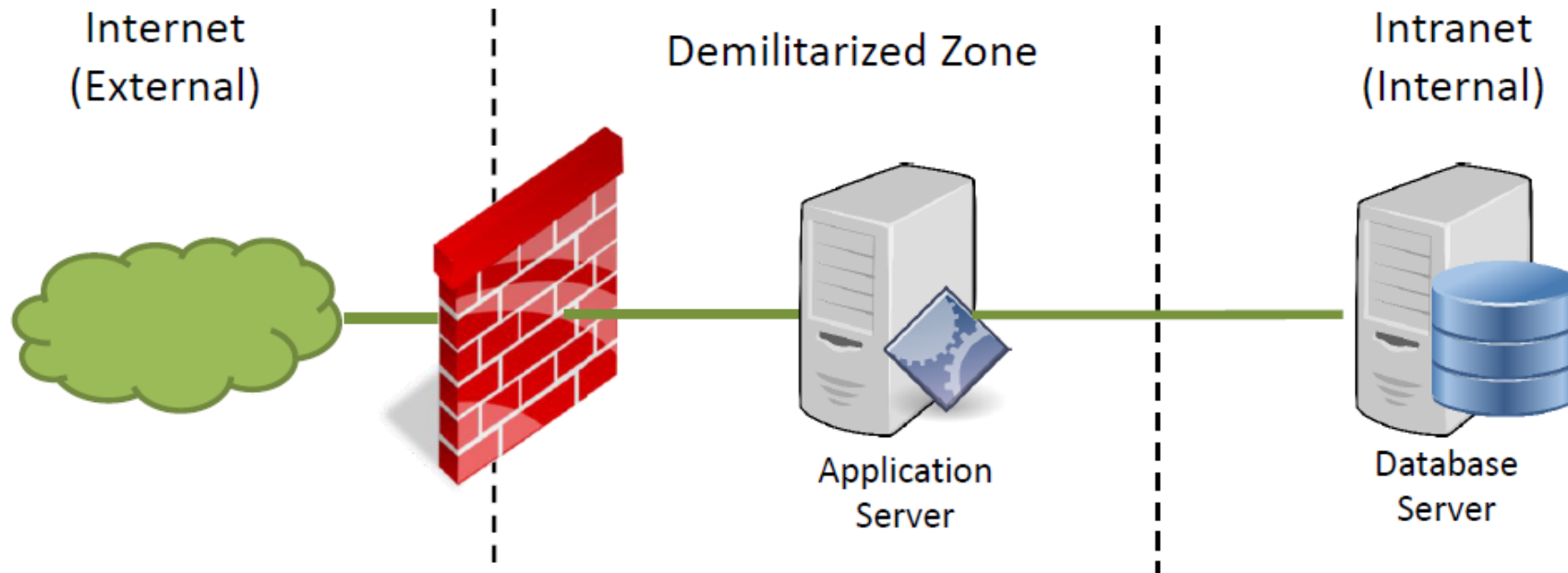
Identify External Dependencies and Techniques

Identify physical topology



Identify physical topology

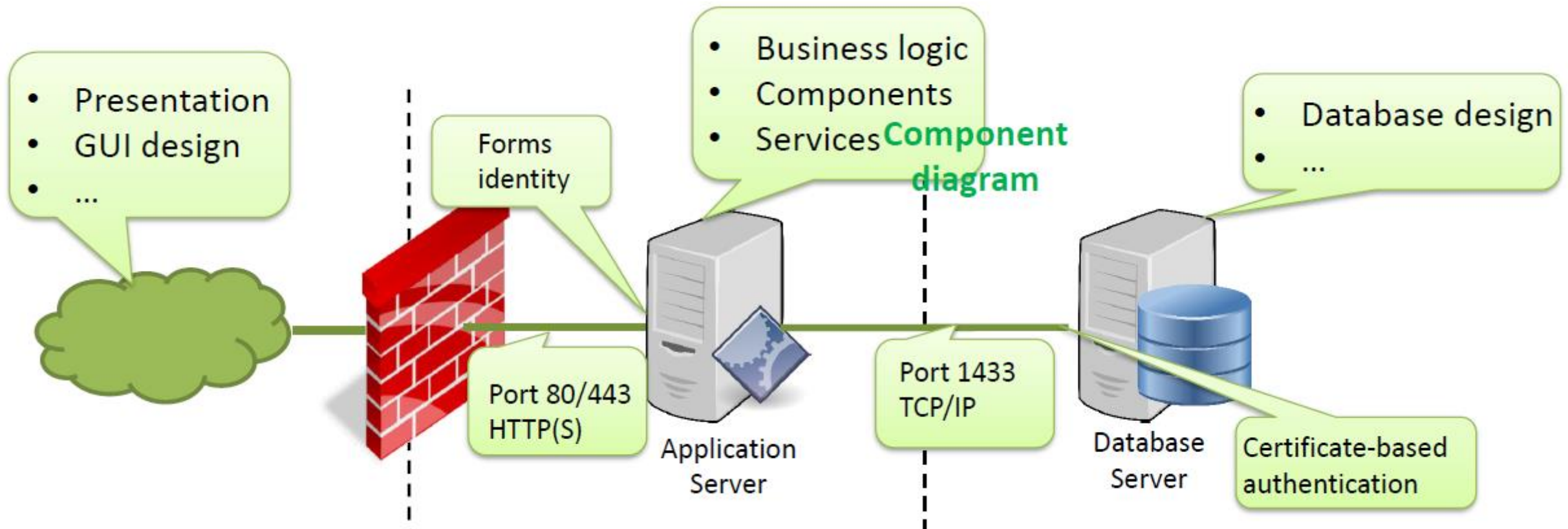
- Where and how the applications will be deployed
- e.g., internal-only, demilitarized zone*, cloud...



How about
Development
environment?

Identify logical topology

- Determine components, services, protocols, ports, etc. that need to be defined, developed and configured in the application
- Identify the identities that will be used in the application and determine how authentication will be designed in the application
 - E.g., forms-based, certificate-based, token-based, biometrics, single sign-on, multifactor, etc.



Identify Human and Non-Human Actors of the system

- Customers, sales agents, system administrators, database administrators, developer, tester, etc.
- A backup process which backs up database periodically
- Any backend process / program of the system should be considered as well



Identify Data Elements

- Product information, customer information, etc.
- Testing data
- Product data / configuration
- Passwords
- Sensitive and non-sensitive data



Generate a Data Access Control Matrix

		User Roles		
		Administrator	Customer	Sales Agent
Data	Customer Data	C, R, U, D	C, R, U, D	C, R, U
	Product Data	C, R, U, D	R	R, U
	Order Data		C, R, U, D	C, R, U, D
	Credit Card Data		C, R, U, D	R, U

Identify External Dependencies and Techniques

- External dependencies
 - Software and hardware components that the software relies on but developed by third parties
 - Examples: credit card processor, third-part backup services
- Techniques (using community resource), including by not limited in
 - Authentication: basic authentication (what you know, what you are, what you have), multifactor,
 - Single Sign-on, OTP, etc.
 - Password management: complexity policy, expiration date, recover/reset strategies, NIST standards, etc.
 - Certification management: PKI, CA, certificate revocation, etc.
 - Flow control: firewalls, proxy, etc.
 - Auditing: Application event logs, syslog, etc.
 - Intrusion Detection System (IDS)
 - Database Security: Database encryption
 - Sandbox

QUESTION AND COMMENTS?

Reference

- <https://docs.microsoft.com/en-us/azure/security/azure-security-threat-modeling-tool-threats>
- https://insights.sei.cmu.edu/sei_blog/2018/12/threat-modeling-12-available-methods.html
- <https://www.first.org/cvss/specification-document>